

The kitchen line, driven in a live browser

examples/hotel/kitchen.bl — a (pipeline ...) clause, and the page that reports it

beam-lisp · 2026-09-18 · three frames from two independent runs

1. What was driven

examples/hotel/kitchen.bl is one file. It declares a line and then serves its own read model:

```
(proc/defserver kitchen
  (pipeline
    (from (arrivals)) ; one order every 60 ms
    (stage :check check-order {:concurrency 2}))
    (batch :pan 8 {:timeout 150})
    (stage :cook cook-pan {:concurrency 4}) ; 320 ms on a burner
    (stage :plate plate-pan)
    (on-lag :block)
    (on-error :requeue)))
```

240 covers. Run as `bl serve examples/hotel/kitchen.bl`, it binds port 4058 and answers on `/`. The page is plain HTML with a one-second meta refresh — one second because the cook is a few hundred milliseconds and a slower frame could miss a worker mid-pan.

2. How it was driven

The server was started from the checkout, waited on with `curl` until it answered 200 (110 s: JIT and namespace loads, not the line), and then opened in a real headless browser. Every frame below is a screenshot of that browser, taken with a full-page capture at a 2× device scale. The line was run twice, from two separate server processes, to check that the outcome repeats.

3. The frames

3.1. Running — 10 seconds in

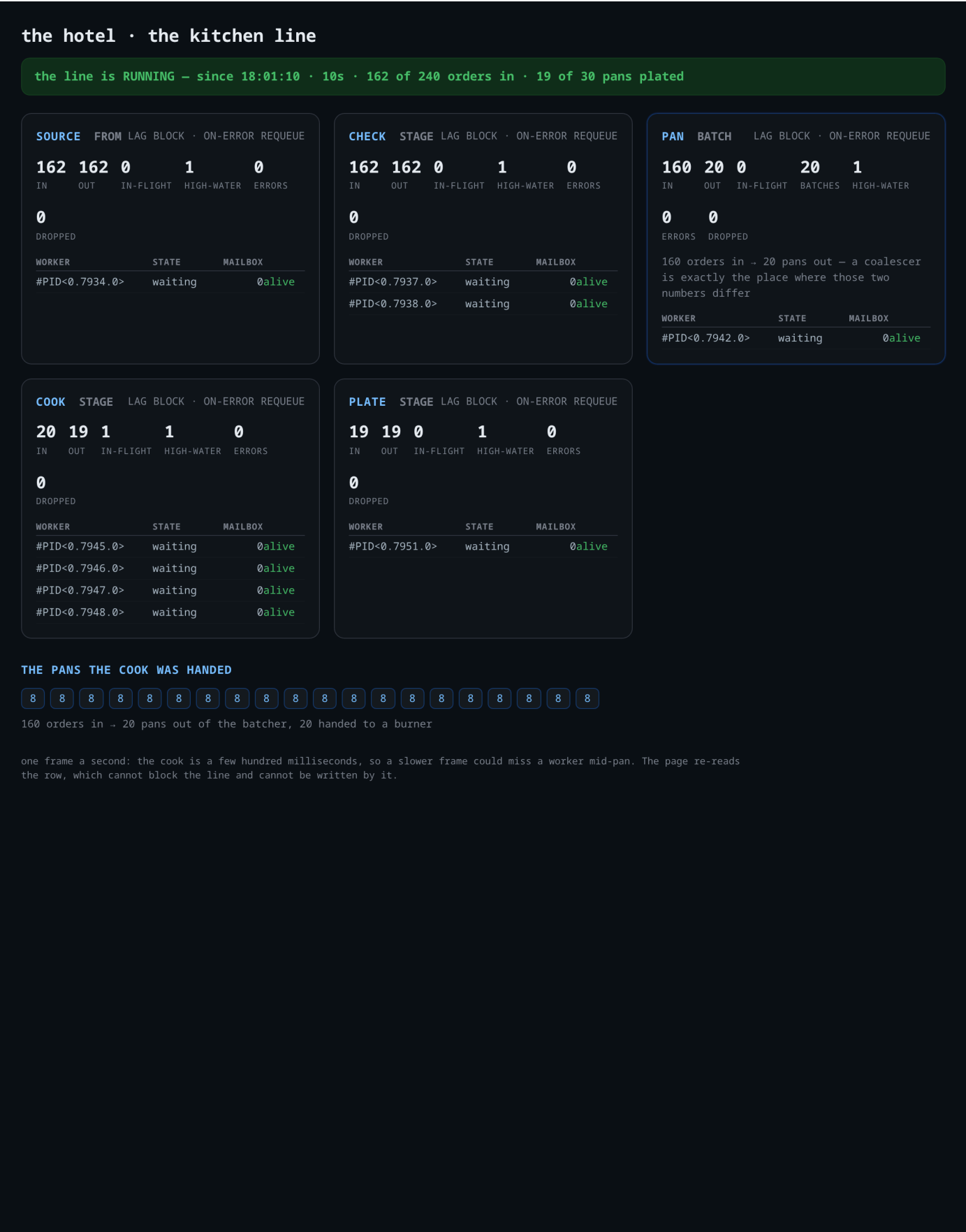


Figure 1: The line is RUNNING: 162 of 240 orders in, 19 of 30 pans plated.

Read the four things this frame proves, in the order they appear:

the policy is on the row. **Every card carries LAG BLOCK · ON-ERROR REQUEUE — the policy actually in force, not the one the declaration asked for. A row that showed throughput without it would be a row nobody could act on. the batcher's arithmetic is visible.** PAN reads IN 160 · OUT 20, while the SOURCE reads 162. The two orders of difference are the two sitting **inside** the coalescer, waiting for a pan to reach eight — and the page says so (240 orders in · 30 pans out – a coalescer is exactly the place where those two numbers differ). SOURCE 162 → PAN 160 → COOK 20 → PLATE 19 are the four different units the same work is counted in. a worker is caught mid-work. **COOK shows IN-FLIGHT 1 with four pids under it — one burner is inside its 320 ms sleep at the instant of the frame. the workers are processes, and they are alive.** Each is listed by pid with its STATE and MAILBOX: waiting and 0alive (zero messages queued). This is the reason the pane reports pids rather than a count.

3.2. Finished — 82 seconds in

the hotel · the kitchen line

SOURCE FROM LAG BLOCK · ON-ERROR REQUEUE

240	240	0	1	0
IN	OUT	IN-FLIGHT	HIGH-WATER	ERRORS

0
DROPPED

CHECK STAGE LAG BLOCK · ON-ERROR REQUEUE

240	240	0	1	0
IN	OUT	IN-FLIGHT	HIGH-WATER	ERRORS

0
DROPPED

PAN **BATCH** **LAG** **BLOCK** **ON-ERROR** **REQUEUE**

240	30	0	30	1
IN	OUT	IN-FLIGHT	BATCHES	HIGH-WATER

0 0
ERRORS DROPPED

COOK STAGE LAG BLOCK · ON-ERROR REQUEUE

30	30	0	1	0
IN	OUT	IN-FLIGHT	HIGH-WATER	ERRORS

0
DROPPED

PLATE	STAGE	LAG	BLOCK	ON-ERROR	REQUEUE
1	1	0	1	0	0
1	2	0	1	0	0
1	3	0	1	0	0
1	4	0	1	0	0
1	5	0	1	0	0
1	6	0	1	0	0
1	7	0	1	0	0
1	8	0	1	0	0
1	9	0	1	0	0
1	10	0	1	0	0
1	11	0	1	0	0
1	12	0	1	0	0
1	13	0	1	0	0
1	14	0	1	0	0
1	15	0	1	0	0
1	16	0	1	0	0
1	17	0	1	0	0
1	18	0	1	0	0
1	19	0	1	0	0
1	20	0	1	0	0
1	21	0	1	0	0
1	22	0	1	0	0
1	23	0	1	0	0
1	24	0	1	0	0
1	25	0	1	0	0
1	26	0	1	0	0
1	27	0	1	0	0
1	28	0	1	0	0
1	29	0	1	0	0
1	30	0	1	0	0
1	31	0	1	0	0
1	32	0	1	0	0
1	33	0	1	0	0
1	34	0	1	0	0
1	35	0	1	0	0
1	36	0	1	0	0
1	37	0	1	0	0
1	38	0	1	0	0
1	39	0	1	0	0
1	40	0	1	0	0
1	41	0	1	0	0
1	42	0	1	0	0
1	43	0	1	0	0
1	44	0	1	0	0
1	45	0	1	0	0
1	46	0	1	0	0
1	47	0	1	0	0
1	48	0	1	0	0
1	49	0	1	0	0
1	50	0	1	0	0
1	51	0	1	0	0
1	52	0	1	0	0
1	53	0	1	0	0
1	54	0	1	0	0
1	55	0	1	0	0
1	56	0	1	0	0
1	57	0	1	0	0
1	58	0	1	0	0
1	59	0	1	0	0
1	60	0	1	0	0
1	61	0	1	0	0
1	62	0	1	0	0
1	63	0	1	0	0
1	64	0	1	0	0
1	65	0	1	0	0
1	66	0	1	0	0
1	67	0	1	0	0
1	68	0	1	0	0
1	69	0	1	0	0
1	70	0	1	0	0
1	71	0	1	0	0
1	72	0	1	0	0
1	73	0	1	0	0
1	74	0	1	0	0
1	75	0	1	0	0
1	76	0	1	0	0
1	77	0	1	0	0
1	78	0	1	0	0
1	79	0	1	0	0
1	80	0	1	0	0
1	81	0	1	0	0

30	30	0	1	0
IN	OUT	IN-FLIGHT	HIGH-WATER	ERRORS

0
DROPPED

THE PANS THE COOK WAS HANDED

Figure 3: Run 2, finished: the same $240 \rightarrow 30$ outcome, from a fresh server process.

The finished frame is the audit. 240 orders in → 30 pans out, all thirty pans size 8, 0 errors, 0 dropped, in-flight 0 at every stage, and every worker still **in the row** — reading dead rather than having been removed from it.

A row that silently got shorter when a worker died would answer “how many cooks do I have” with a shrug; this one answers “which cook stopped”.

4. What was checked

claim	how the page shows it	verdict
240 covers, one every 60 ms	SOURCE 240 in · 240 out · 0 errors	ok
pans are always the eight the declaration says	THE PANS THE COOK WAS HANDED: 8 × 30, and 240 = 30 × 8	ok
the batcher fires the source’s pace, not the clock	PAN 240 in → 30 batches; the 150 ms timeout only matters at the tail	ok
the cook is busy, not idle	COOK IN-FLIGHT 1 at 10 s; STATE waiting on all four when drained	ok
nothing is lost or duplicated	0 dropped on all five stages, in 240 = out 240 at the source	ok
the same run repeats	two separate server processes, same numbers, 240 → 30	ok
a dead worker stays visible	STATE dead on all nine pids, none removed from the row	ok

5. Running it

```
cd /home/user/code/undefine/beam-lisp
./bin/bl serve examples/hotel/kitchen.bl # binds 4058; ~110 s to first answer
curl -s http://127.0.0.1:4058/ | head -c 200 # the same facts as the page, as HTML
```

Two notes for anyone repeating this:

the demo needs the application to start, and the application cannot start on a floor that is a previous generation of priv/boot/. That was the state of this checkout (BUG-089): the boot tier had moved and the committed floor had not. It was repaired by rebuilding the tier as a unit — bl build’s driver, run against the staged floor as its compiler — and re-publishing with bl seed. Every frame here is from after that repair. the line starts when the server starts, so a frame taken right after the first 200 is a frame of a line in its first seconds. The running frame above was taken that way, deliberately.